Laravel Backend API Best Practices Guide

Table of Contents

Design System & Architecture

RESTful API Design

Resource Naming Conventions - Use plural nouns for resources: `/api/users`, `/api/products` - Use hyphens for multi-word resources: `/api/user-profiles` - Avoid deep nesting (max 2 levels): `/api/users/{id}/posts` , `/api/users/{id}/posts/{id}/comments/{id}`

HTTP Methods - `GET` - Retrieve resources (idempotent) - `POST` - Create new resources - `PUT/PATCH` - Update existing resources - `DELETE` - Remove resources

Status Codes - `200 OK` - Successful GET, PUT, PATCH - `201 Created` - Successful POST - `204 No Content` - Successful DELETE - `400 Bad Request` - Validation errors - `401 Unauthorized` - Authentication required - `403 Forbidden` - Authenticated but not authorized - `404 Not Found` - Resource doesn't exist - `422 Unprocessable Entity` - Validation failed - `500 Internal Server Error` - Server errors

Repository Pattern

```php
// App/Repositories/UserRepository.php
interface UserRepositoryInterface {
    public function all();
    public function find($id);
    public function create(array $data);
    public function update($id, array $data);
    public function delete($id);
}

class UserRepository implements UserRepositoryInterface {
    protected $model;

    public function __construct(User $model) {
        $this->model = $model;
```

```
    }

    public function all() {
        return $this->model->all();
    }
}

// Register in AppServiceProvider
public function register() {
    $this->app->bind(
        UserRepositoryInterface::class,
        UserRepository::class
    );
}
```

Service Layer Pattern

```
// App/Services/UserService.php
class UserService {
    protected $userRepository;

    public function __construct(UserRepositoryInterface $userRepository) {
        $this->userRepository = $userRepository;
    }

    public function createUser(array $data) {
        // Business logic here
        $data['password'] = bcrypt($data['password']);
        return $this->userRepository->create($data);
    }
}
```

API Versioning

URL Versioning (Recommended)

```
Route::prefix('v1')->group(function () {
    Route::apiResource('users', UserController::class);
});

Route::prefix('v2')->group(function () {
    Route::apiResource('users', V2\UserController::class);
});
```

Response Structure

Consistent JSON Response Format

```php
// App/Traits/ApiResponse.php
trait ApiResponse {
    protected function successResponse($data, $message = null, $code = 200) {
        return response()->json([
            'status' => 'success',
            'message' => $message,
            'data' => $data
        ], $code);
    }

    protected function errorResponse($message, $code = 400, $errors = null) {
        return response()->json([
            'status' => 'error',
            'message' => $message,
            'errors' => $errors
        ], $code);
    }
}
```

Security Best Practices

Authentication & Authorization

Use Laravel Sanctum for API Authentication

```php
// config/sanctum.php
'expiration' => 60 * 24, // Token expiration in minutes

// Controller
public function login(Request $request) {
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required'
    ]);

    if (!Auth::attempt($credentials)) {
        return response()->json(['message' => 'Invalid credentials'], 401);
    }

    $user = Auth::user();
    $token = $user->createToken('api-token')->plainTextToken;

    return response()->json([
        'token' => $token,
        'user' => $user
```

```
    ]);
}
```

Implement Role-Based Access Control (RBAC)

```
// Use Laravel Gates or Policies
// app/Policies/PostPolicy.php
public function update(User $user, Post $post) {
    return $user->id === $post->user_id;
}

// In Controller
public function update(Request $request, Post $post) {
    $this->authorize('update', $post);
    // Update logic
}
```

Input Validation & Sanitization

Always Use Form Requests

```
// app/Http/Requests/StoreUserRequest.php
class StoreUserRequest extends FormRequest {
    public function authorize() {
        return true;
    }

    public function rules() {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8|confirmed',
            'phone' => 'nullable|regex:/^[0-9]{10}$/'
        ];
    }

    public function messages() {
        return [
            'email.unique' => 'This email is already registered',
        ];
    }
}
```

Prevent Mass Assignment Vulnerabilities

```
// In Model
protected $fillable = ['name', 'email', 'password'];
// OR
```

```php
protected $guarded = ['id', 'is_admin'];
```

SQL Injection Prevention

Always Use Eloquent ORM or Query Builder

```php
//  CORRECT - Parameterized queries
User::where('email', $email)->first();
DB::table('users')->where('email', $email)->get();

//  AVOID - Raw queries without binding
DB::select("SELECT * FROM users WHERE email = '$email'");

//  If raw SQL needed, use parameter binding
DB::select('SELECT * FROM users WHERE email = ?', [$email]);
```

Cross-Site Scripting (XSS) Protection

```php
// Escape output in Blade templates (automatic)
{{ $user->name }} // Escaped
{!! $user->bio !!} // Not escaped - use carefully

// For API responses, validation handles this
'bio' => 'required|string|max:1000'
```

CORS Configuration

```php
// config/cors.php
return [
    'paths' => ['api/*'],
    'allowed_methods' => ['*'],
    'allowed_origins' => [env('FRONTEND_URL', 'http://localhost:3000')],
    'allowed_origins_patterns' => [],
    'allowed_headers' => ['*'],
    'exposed_headers' => [],
    'max_age' => 0,
    'supports_credentials' => true,
];
```

Rate Limiting

```php
// app/Providers/RouteServiceProvider.php
protected function configureRateLimiting() {
    RateLimiter::for('api', function (Request $request) {
        return Limit::perMinute(60)->by($request->user()?->id ?: $request->ip());
    });
```

```
    RateLimiter::for('login', function (Request $request) {
        return Limit::perMinute(5)->by($request->ip());
    });
}

// Apply to routes
Route::middleware(['throttle:api'])->group(function () {
    // Your routes
});
```

Environment Variables Security

```
# .env
APP_DEBUG=false # Never true in production
APP_ENV=production

# Use strong keys
APP_KEY=base64:...

# Never commit .env file
# Add to .gitignore
.env
.env.backup
```

HTTPS & Secure Headers

```
// app/Http/Middleware/SecureHeaders.php
public function handle($request, Closure $next) {
    $response = $next($request);

    $response->headers->set('X-Frame-Options', 'DENY');
    $response->headers->set('X-Content-Type-Options', 'nosniff');
    $response->headers->set('X-XSS-Protection', '1; mode=block');
    $response->headers->set('Strict-Transport-Security', 'max-age=31536000');

    return $response;
}
```

Password Security

```
// Use bcrypt (Laravel default)
'password' => bcrypt($request->password);

// Or Hash facade
use Illuminate\Support\Facades\Hash;
Hash::make($request->password);
```

```php
// Verification
Hash::check($plainPassword, $hashedPassword);
```

Performance Optimization

Database Query Optimization

Eager Loading (N+1 Problem)

```php
//  BAD - N+1 queries
$users = User::all();
foreach ($users as $user) {
    echo $user->posts->count(); // Query per user
}

//  GOOD - 2 queries total
$users = User::with('posts')->get();
foreach ($users as $user) {
    echo $user->posts->count();
}
```

Lazy Eager Loading

```php
$users = User::all();
$users->load('posts', 'comments');
```

Select Specific Columns

```php
// Only get needed columns
User::select('id', 'name', 'email')->get();
```

Chunking Large Datasets

```php
User::chunk(100, function ($users) {
    foreach ($users as $user) {
        // Process user
    }
});
```

Caching Strategies

Query Result Caching

```php
$users = Cache::remember('users', 3600, function () {
    return User::all();
});

// Cache with tags (Redis/Memcached)
```

```php
Cache::tags(['users', 'active'])->put('key', $value, 3600);
Cache::tags(['users'])->flush();
```

API Response Caching

```php
// Use Laravel's Response Cache package
Route::get('/users', [UserController::class, 'index'])
    ->middleware('cache.response:3600');
```

Model Caching

```php
// In Model
protected $cacheFor = 3600;

public function getCachedAttribute() {
    return Cache::remember("user.{$this->id}.attribute", 3600, function () {
        return $this->expensiveOperation();
    });
}
```

Database Indexing

```php
// In migration
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('email')->unique(); // Automatic index
    $table->string('username');
    $table->timestamps();

    // Add custom indexes
    $table->index('username');
    $table->index(['last_name', 'first_name']);
});
```

Queue Jobs for Heavy Tasks

```php
// Create job
php artisan make:job ProcessLargeFile

// Job class
class ProcessLargeFile implements ShouldQueue {
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function handle() {
        // Heavy processing
    }
}
```

```php
// Dispatch
ProcessLargeFile::dispatch($data);
```

Use Redis for Sessions & Cache

```
// .env
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis
```

API Resource Transformation

```php
// app/Http/Resources/UserResource.php
class UserResource extends JsonResource {
    public function toArray($request) {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'posts' => PostResource::collection($this->whenLoaded('posts')),
            'created_at' => $this->created_at->format('Y-m-d'),
        ];
    }
}

// In Controller
return UserResource::collection($users);
```

Pagination

```php
// Simple pagination
$users = User::paginate(15);

// Cursor pagination (better for large datasets)
$users = User::cursorPaginate(15);

// API response
return UserResource::collection($users);
```

Database Connection Pooling

```php
// config/database.php
'mysql' => [
    'read' => [
        'host' => env('DB_READ_HOST', '127.0.0.1'),
    ],
    'write' => [
```

```
        'host' => env('DB_WRITE_HOST', '127.0.0.1'),
    ],
    'sticky' => true, // Use write connection for reads after write
],
```

Code Organization

Folder Structure

```
app/
  Http/
     Controllers/
        Api/
           V1/
     Requests/
     Resources/
     Middleware/
  Models/
  Repositories/
     Interfaces/
  Services/
  Traits/
  Exceptions/
```

Single Responsibility Principle

Controllers should be thin

```php
//  BAD - Fat controller
public function store(Request $request) {
    $validated = $request->validate([...]);
    $user = User::create($validated);
    Mail::to($user)->send(new Welcome($user));
    event(new UserRegistered($user));
    return response()->json($user, 201);
}

//  GOOD - Delegated to service
public function store(StoreUserRequest $request, UserService $service) {
    $user = $service->createUser($request->validated());
    return new UserResource($user);
}
```

Use Eloquent Scopes

```php
// In Model
public function scopeActive($query) {
    return $query->where('active', true);
}

public function scopeRecent($query) {
    return $query->where('created_at', '>=', now()->subDays(7));
}

// Usage
User::active()->recent()->get();
```

Use Observers for Model Events

```php
// app/Observers/UserObserver.php
class UserObserver {
    public function created(User $user) {
        Mail::to($user)->send(new WelcomeEmail($user));
    }

    public function deleting(User $user) {
        $user->posts()->delete();
    }
}

// Register in AppServiceProvider
User::observe(UserObserver::class);
```

Custom Exceptions

```php
// app/Exceptions/InsufficientFundsException.php
class InsufficientFundsException extends Exception {
    public function render($request) {
        return response()->json([
            'error' => 'Insufficient funds',
            'code' => 'INSUFFICIENT_FUNDS'
        ], 422);
    }
}
```

Testing Strategies

Feature Tests

```php
// tests/Feature/UserApiTest.php
class UserApiTest extends TestCase {
    use RefreshDatabase;

    public function test_can_create_user() {
        $response = $this->postJson('/api/users', [
            'name' => 'John Doe',
            'email' => 'john@example.com',
            'password' => 'password123',
            'password_confirmation' => 'password123'
        ]);

        $response->assertStatus(201)
                 ->assertJsonStructure(['data' => ['id', 'name', 'email']]);

        $this->assertDatabaseHas('users', [
            'email' => 'john@example.com'
        ]);
    }

    public function test_requires_authentication() {
        $response = $this->getJson('/api/users');
        $response->assertStatus(401);
    }
}
```

Unit Tests

```php
// tests/Unit/UserServiceTest.php
class UserServiceTest extends TestCase {
    public function test_creates_user_with_hashed_password() {
        $service = new UserService(new UserRepository(new User));

        $user = $service->createUser([
            'name' => 'Test',
            'email' => 'test@test.com',
            'password' => 'plaintext'
        ]);

        $this->assertNotEquals('plaintext', $user->password);
        $this->assertTrue(Hash::check('plaintext', $user->password));
    }
}
```

Database Factories & Seeders

```php
// database/factories/UserFactory.php
public function definition() {
    return [
        'name' => fake()->name(),
        'email' => fake()->unique()->safeEmail(),
        'password' => bcrypt('password'),
    ];
}

// Usage in tests
$user = User::factory()->create();
$users = User::factory()->count(10)->create();
```

Tips & Tricks

API Documentation

Use Laravel API Documentation Packages - Scribe: `composer require --dev knuckleswtf/scribe` - L5-Swagger: `composer require darkaonline/l5-swagger`

Error Handling

```php
// app/Exceptions/Handler.php
public function render($request, Throwable $exception) {
    if ($request->is('api/*')) {
        if ($exception instanceof ModelNotFoundException) {
            return response()->json(['error' => 'Resource not found'], 404);
        }

        if ($exception instanceof ValidationException) {
            return response()->json([
                'error' => 'Validation failed',
                'errors' => $exception->errors()
            ], 422);
        }

        return response()->json([
            'error' => 'Server error',
            'message' => app()->environment('local') ? $exception->getMessage() : 'Something
        ], 500);
    }
```

```
    return parent::render($request, $exception);
}
```

Logging Best Practices

```
use Illuminate\Support\Facades\Log;

// Different log levels
Log::emergency('System is down');
Log::alert('Action must be taken immediately');
Log::critical('Critical conditions');
Log::error('Runtime errors');
Log::warning('Warning messages');
Log::notice('Normal but significant');
Log::info('Informational messages');
Log::debug('Debug messages');

// Add context
Log::info('User registered', ['user_id' => $user->id]);
```

Use UUID Instead of Incremental IDs

```
// In migration
$table->uuid('id')->primary();

// In Model
use Illuminate\Database\Eloquent\Concerns\HasUuids;

class User extends Model {
    use HasUuids;
}
```

API Middleware Stack

```
// routes/api.php
Route::middleware(['auth:sanctum', 'throttle:api', 'log.api'])
    ->prefix('v1')
    ->group(function () {
        Route::apiResource('users', UserController::class);
    });
```

Environment-Specific Configuration

```
// config/app.php
'debug' => env('APP_DEBUG', false),

// Use config() instead of env() in code
```

```php
if (config('app.debug')) {
    // Debug logic
}
```

Database Transactions

```php
use Illuminate\Support\Facades\DB;

DB::transaction(function () {
    $user = User::create($userData);
    $user->profile()->create($profileData);
    $user->sendWelcomeEmail();
});

// Manual transaction control
DB::beginTransaction();
try {
    // Operations
    DB::commit();
} catch (\Exception $e) {
    DB::rollBack();
    throw $e;
}
```

Soft Deletes

```php
// In Model
use Illuminate\Database\Eloquent\SoftDeletes;

class Post extends Model {
    use SoftDeletes;
}

// In migration
$table->softDeletes();

// Usage
$post->delete(); // Soft delete
$post->forceDelete(); // Permanent delete
Post::withTrashed()->get(); // Include soft deleted
```

Query Performance Monitoring

```php
// In AppServiceProvider
if (app()->environment('local')) {
    DB::listen(function ($query) {
        if ($query->time > 100) { // Log slow queries
```

```
            Log::warning('Slow query', [
                'sql' => $query->sql,
                'time' => $query->time
            ]);
        }
    });
}
```

API Health Check Endpoint

```
Route::get('/health', function () {
    return response()->json([
        'status' => 'ok',
        'timestamp' => now(),
        'services' => [
            'database' => DB::connection()->getDatabaseName() ? 'up' : 'down',
            'cache' => Cache::has('health-check') ? 'up' : 'down',
        ]
    ]);
});
```

Deployment Checklist

- ☐ Set `APP_ENV=production`
- ☐ Set `APP_DEBUG=false`
- ☐ Generate new `APP_KEY`
- ☐ Configure proper database credentials
- ☐ Set up Redis for caching
- ☐ Enable HTTPS/SSL
- ☐ Configure CORS properly
- ☐ Set up queue workers
- ☐ Configure log rotation
- ☐ Run `php artisan config:cache`
- ☐ Run `php artisan route:cache`
- ☐ Run `php artisan view:cache`
- ☐ Set up monitoring (Laravel Telescope, Horizon)
- ☐ Configure backup strategy
- ☐ Set up automated deployments
- ☐ Performance testing

Additional Resources

- **Laravel Documentation**: https://laravel.com/docs

- **Laravel API Tutorial**: https://laravel.com/docs/sanctum
- **REST API Best Practices**: https://restfulapi.net/
- **Laravel Testing**: https://laravel.com/docs/testing
- **Security Best Practices**: https://owasp.org/www-project-top-ten/

Document Version: 1.0
Last Updated: November 2025
Author: Laravel Backend API Development Guide